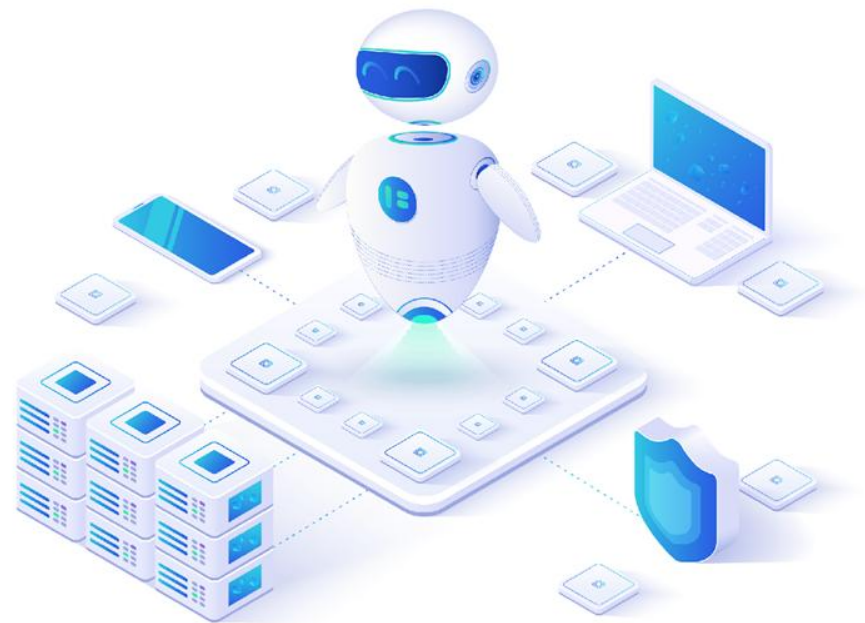




생성형 AI 플랫폼 AI 워크플로우 및 Flowise 교육

제논 | 2026.04

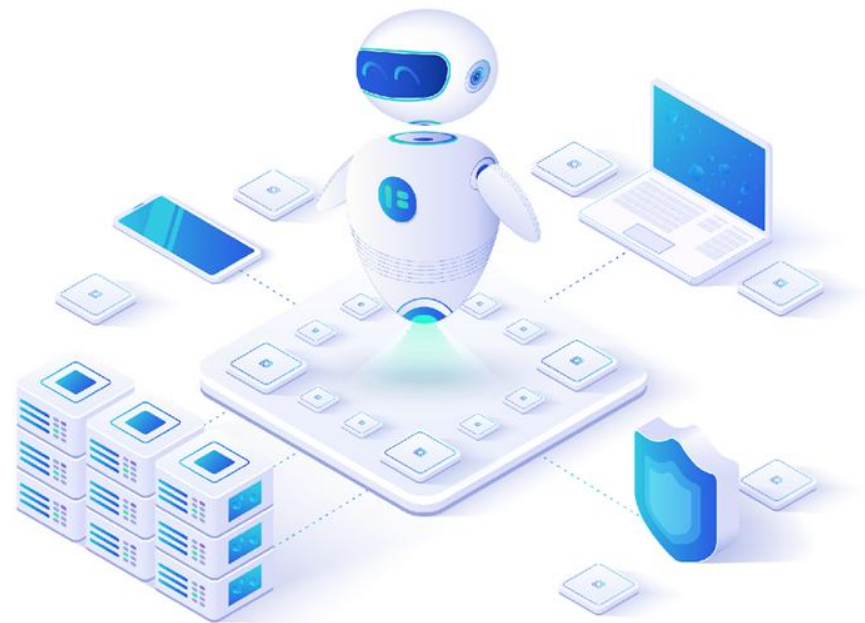


INDEX

- I. 워크플로우와 Flowise
- II. 워크플로우 제작 (기초)
- III. 워크플로우 제작 (심화)
- IV. MCP
- V. 참고자료

I 워크플로우와 Flowise

- | | |
|----------------|---|
| 1. 워크플로우 | 4 |
| 2. Flowise | 5 |
| 3. 워크플로우 활용 사례 | 7 |



1. 워크플로우

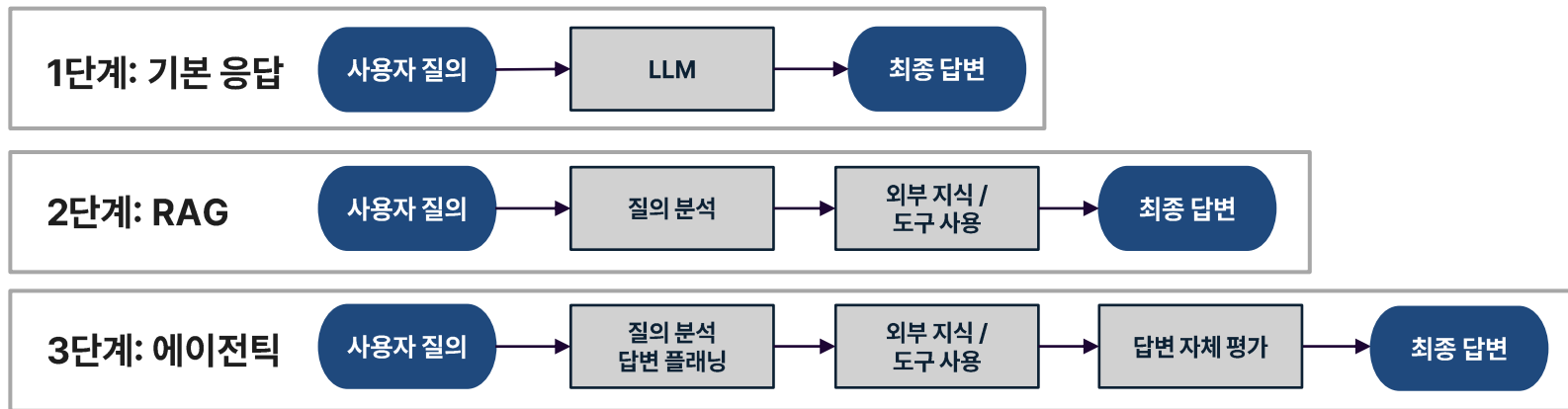
워크플로우 & AI 워크플로우

워크플로우

- 특정 업무 (목표) 에 대한 정보를 처리하는 절차, 작업 흐름

AI 워크플로우

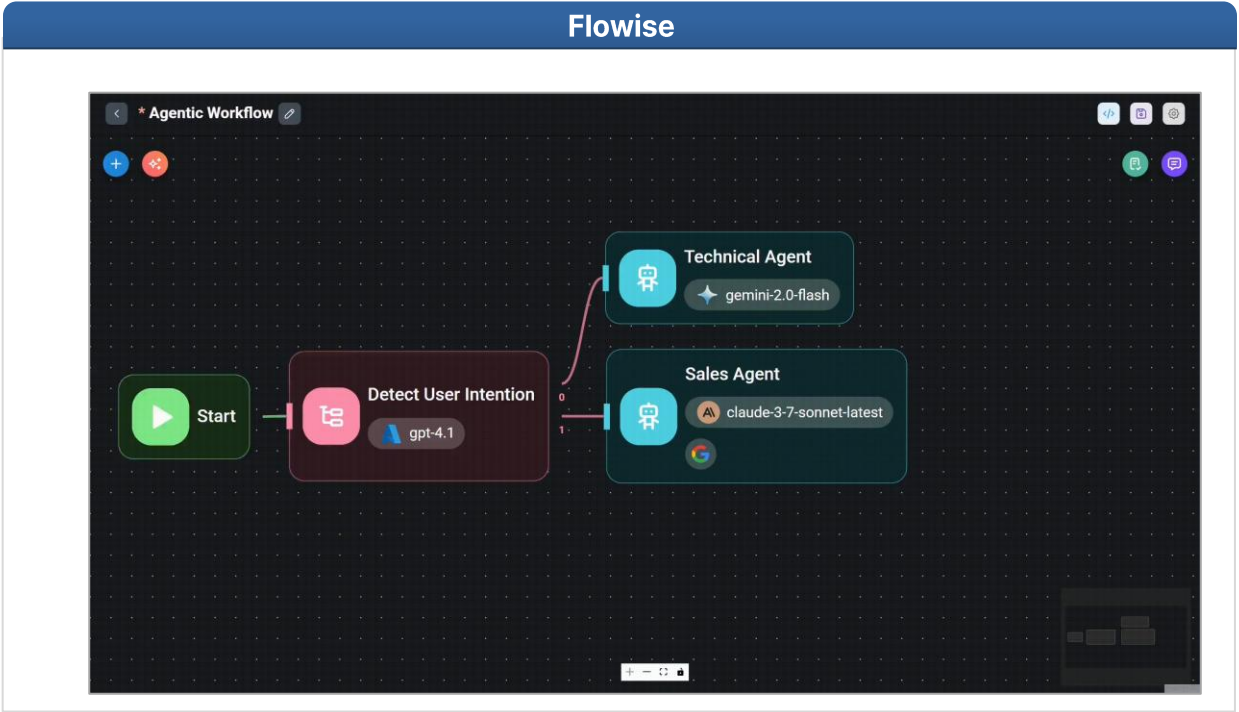
- AI(LLM), 프롬프트, 데이터 소스, 도구들을 논리적으로 연결하여 작업을 수행하는 파이프라인



2. Flowise

Flowise 개요

복잡한 LLM 워크플로우를 시각적 노드로 설계하고 배포할 수 있는 오픈소스 로우코드/노코드 플랫폼



주요 기능

노코드 기반 LLM 에이전트 구성

- 드래그 앤 드롭으로 에이전트 플로우를 시각적으로 구성

멀티 에이전트 및 적합한 LLM 모델 설정

- 여러 에이전트가 협업하는 복잡한 워크플로우 설정 가능
- 각 에이전트 별 LLM 설정 가능

Tool + LLM 자동 조합

- LLM이 판단해 적절한 Tool을 선택/실행할 수 있도록 구성 가능

Agent 기반의 분기 처리

- LLM이 판단해 적합한 분기로 진입 가능

서브 플로우 호출

- 워크플로우 내에서 다른 에이전트 호출 가능

2. Flowise

Flowise 기능

LangChain 코드를 시각적 노드로 구조화하여 워크플로우를 간결하게 구현

LangChain

```

Python

import os
import weaviate
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_weaviate.vectorstores import WeaviateVectorStore
from langchain_chains import create_retrieval_chain
from langchain_chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

# 0. API 키 및 Weaviate 설정
os.environ["OPENAI_API_KEY"] = "sk-..."
WEAVIATE_URL = "https://your-cluster-url.weaviate.network" # Weaviate 클러스터 URL
WEAVIATE_API_KEY = "your-weaviate-api-key" # Weaviate API 키

# 1. Weaviate 클라이언트 생성 (v4 클라이언트 필요)
# 이미 데이터가 저장된 클러스터(Cloud) 혹은 로컬 서버에 접속합니다.
client = weaviate.connect_to_wcs(
    cluster_url=WEAVIATE_URL,
    auth_credentials=weaviate.auth.AuthApiKey(WEAVIATE_API_KEY),
    # Weaviate OpenAI 모듈을 사용하려면 Weaviate가 내부적으로 임베딩을 처리할 수 있습니다.
    headers={"X-OpenAI-API-Key": os.environ["OPENAI_API_KEY"]}
)

# 2. 기존 데이터 저장소 생성 (Connect)
# 'index_name'은 Weaviate의 데이터 저장소인 클래스 이름(예: 'Article', 'Document')이며
vectorstore = WeaviateVectorStore(
    client=client,
    index_name="LangChain_Article", # <--> 중요! 이미 데이터가 있는 클래스명
    text_key="text", # <--> 중요! 텍스트가 저장된 속성(필드) 이름
    embedding=OpenAIEmbeddings()
)

retriever = vectorstore.as_retriever()

# 3. RAG 체인 생성 (이전과 동일)
llm = ChatOpenAI(model="gpt-3.5-turbo")

prompt = ChatPromptTemplate.from_template("""
아래 문단(context)을 사용하여 질문에 답변해 주세요:
<context>
{context}
</context>

질문: {input}
""")

document_chain = create_stuff_documents_chain(llm, prompt)
retrieval_chain = create_retrieval_chain(retriever, document_chain)

# 4. 실행
try:
    response = retrieval_chain.invoke({"input": "Weaviate의 장점이 뭐야?"})
    print(response["answer"])
finally:
    client.close() # 리소스 해제를 위해 클라이언트 종료
    
```



Flowise

```

graph TD
    Start([Start]) --> RAGAgent[RAG Agent]
    
```

GenOS 워크플로우 활용 사례

1 사규 및 컴플라이언스 RAG

- 사규와 관련 법령 RAG
- 법령 변경 시 사규에 미칠 영향 검토 가능

2 데이터 분석 에이전트

- 사용자 질의를 분석하여 Text2SQL
- 복수 테이블 Join 및 시각화
- Human In The Loop

3 리서치 에이전트

- 사용자 질의를 분석하여 적절한 외부 검색
- 외부 검색 결과 시각화 필요 시 수행

4 작성 가이드 기반 첨삭 어시스턴트

- 사내 작성 가이드에 기반한 문서 내용 교정
- 첨삭 사유 및 근거 제시
- 파일 업로드 지원

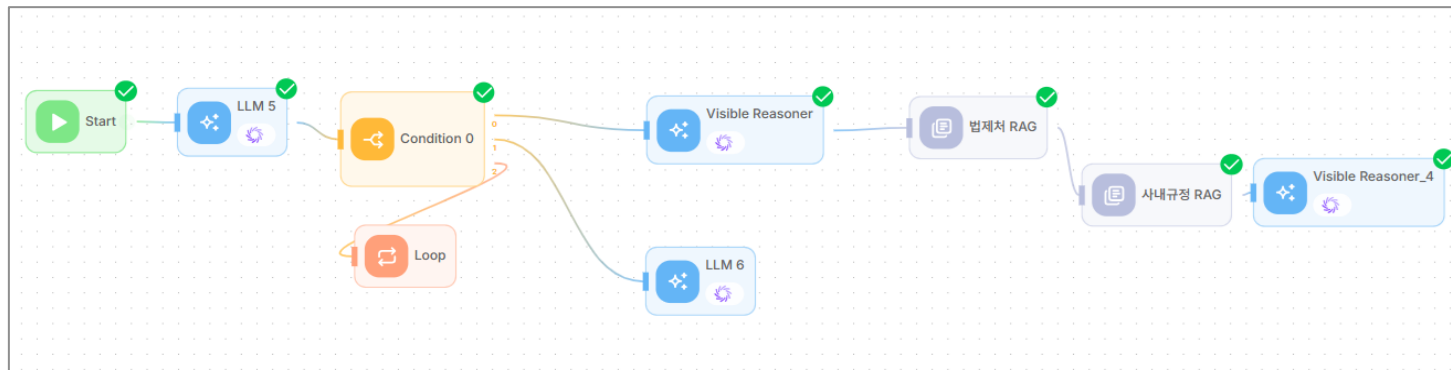
5 제약 리서치 에이전트

- 리서치 플래너가 질의에 필요한 에이전트만 선택하여 답변 생성 진행
- 외부 MCP 도구 활용

사규 및 컴플라이언스 RAG

사규 및 법령 벡터DB 동시 검색으로 개정 사항 발생 시 관련 규정 간 영향 파악

사규 및 컴플라이언스 RAG



사규 및 컴플라이언스 RAG

사규 및 법령 벡터DB 동시 검색으로 개정 사항 발생 시 관련 규정 간 영향 파악

사규 및 컴플라이언스 RAG 질의 답변 예시

입력

고객의 권리 강화를 위해 '개인정보 열람 요구 시 7일 이내 처리'를 의무화하는 법안이 논의 중입니다.
현재 우리 회사의 관련 규정은 어떻게 되나요?



출력

원본 :질문에 대한 핵심 결론

현재 회사는 개인정보 열람 요구에 대해 **10일 이내**에 열람을 제공하도록 규정하고 있으며, 7일 이내 처리 의무화는 아직 사내 규정에 반영되지 않았습니다.

개인정보 열람에 대한 처리 기한

제27조(개인정보의 열람) ①에 따라, "회사는 정보주체로부터 개인정보의 열람을 요구 받았을 때 **10일 이내**에 정보주체가 해당 개인정보를 열람할 수 있도록 하여야 한다."...

→ 현재 사내 규정은 열람 요구 처리 기간을 **10일로** 명시하고 있으며, 7일로 단축하는 내용은 포함되어 있지 않다.

[출처: '개인정보보호규정, 제27조(개인정보의 열람)']

최종 결론

현재 내부 규정은 개인정보 열람 요구에 대해 **10일 이내** 처리하도록 명시하고 있으며, 7일 이내 처리 의무화는 아직 규정에 반영되지 않았다. 따라서 향후 법안이 통과될 경우, 사내 규정 개정을 통해 처리 기간을 7일로 단축하는 검토가 필요하다.

데이터 분석 에이전트

자연어 질의를 쿼리로 바꿔 복수의 테이블을 Join하여 현업 분석 로직을 반영한 데이터 분석

데이터 분석 에이전트

입력

202501~202505 건강월초가 10만 이상인 건수가 가장 높은 지역단 10개 뽑아줘.

Text2SQL

2. 실행된 SQL 쿼리

```

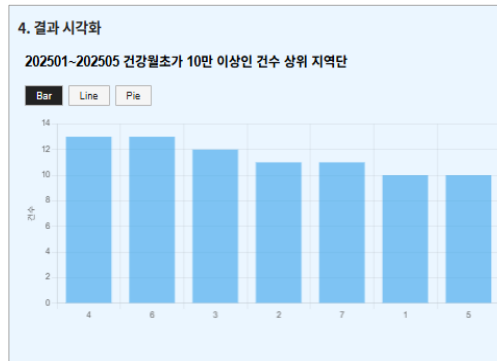
SELECT G00001.CLOT_OF_PPL_HD AS 지역인구정보,
COUNT(DISTINCT NOCNT001.CCNT_HD) AS 인구수
FROM NOCNT001
JOIN G00001 ON G00001.CCNT_HD = NOCNT001.CLOT_CBLT_HD
WHERE NOCNT001.FINLNM BETWEEN 202501 AND 202505
AND NOCNT001.CCNT_STAT_CD != 0
AND NOCNT001.MPCL_PFM = NOCNT001.FINL_PFM
AND NOCNT001.PPOT_JCLUP_CD = 0
AND NOCNT001.PPOT_JCLUP_CD = 1
AND NOCNT001.MPCL_PFM >= 100000
GROUP BY G00001.CLOT_OF_PPL_HD
ORDER BY 인구수 DESC
LIMIT 10;
    
```

3. SQL 쿼리 결과 (쿼리 실행 결과)

지역인구정보	인구
4	13
6	13
3	12
2	11
7	11
1	10
5	10

참고: SQL 실행 결과는 총 7개의 지역단 정보만 포함되어 있습니다. 사용자의 요청은 상위 10개를 요청했으나, 제공된 결과는 최대 7개까지만 존재합니다. 추가적인 데이터는 없으므로, 이 결과를 그대로 표시합니다.

시각화



시작

스키마
정보

질의 분석

쿼리 생성, 검증

시각화 &
최종답변

종료

3. 워크플로우 활용 사례

리서치 에이전트

최종 답변을 도출하고자 작업 계획을 세운 뒤 이를 기반으로 체계적으로 리서치 수행

리서치 에이전트

최근 3년간 금융 산업에서 생성형 AI 도입 사례와 그 영향을 연도별로 테이블로 정리하고, 주요 변화 요인을 요약해 드리겠습니다.

연도	생성형 AI 도입 사례 및 활용 내용	주요 영향 및 변화
2022년	- 12월 OpenAI 챗GPT 출시 및 금융권 관심 급증 - 초기에는 내부 직원 업무 지원, 문서 요약, 보고서 작성 보조에 활용	- 생성형 AI 기술 첫 등장, 금융업계의 디지털 전환 가속화 - 업무효율성과 정확성 향상 가능성 인식 시작
2023년	- KB금융 'KB-GPT' 업무지원 AI병커 도입 - 금융권 AI 고객상담센터(AICC)에서 챗봇 고도화 및 고객 서비스 개선 - 미국 JP모건체이스 400여개 업무에 생성형 AI 적용	- 금융기관 내부보고서 작성, 상품 홍보문구 작성 등 업무 효율성 증대 - 고객응대 자동화로 고객만족도 및 신뢰도 증대 - 자산관리, 재무예측, 이상거래 탐지 등 업무 다양화
2024년	- 캐피탈원 등 사기 탐지 시스템에 생성형 AI 도입 - 국내 NH농협, 신한은행 등 대출심사 및 추천서비스에 AI 활용 확대 - 신한라이프 등 보험사 AI 조직 구축 및 DX 추진 - JP모건체이스 400개 이상 AI 활용 사례 확대	- 사기거래 탐지 정확도 향상 및 리스크 관리 강화 - AI 금융상품 추천, 기업 대출 심사 등 금융서비스 다양화 - 금융상품 개발 및 고객 맞춤형 서비스 강화 - 업무 자동화와 고객 맞춤형 자산관리 고도화
2025년	- 금융권 AI 플랫폼 구축 통한 상호-오픈소스 AI 전략적 활용 - 금융산업 내 AI 거버넌스 강화, 규제 개선 필요성 부각 - 생성형 AI 활용 금융산업 혁신 가속화 전망 - AI 기반 리스크 관리 및 고객 상담 고도화 지속	- AI 도입 범위 확대 및 디지털 금융 채널 변화 가속화 - AI 영업 리스크, 정보보호, 윤리 문제 대응 체계 마련 필요 - 디지털금융 개인화, 자동화 심화, 금융산업 경쟁력 강화 - 고객 서비스 품질 향상 및 금융사고 예방 가능

최근 3년간 금융 산업에서 생성형 AI 도입 사례와 그 영향을 연도별로 테이블로 정리하고, 주요 변화 요인을 요약해 드리겠습니다.

연도	생성형 AI 도입 사례 및 활용 내용	주요 영향 및 변화
2022년	- 12월 OpenAI 챗GPT 출시 및 금융권 관심 급증 - 초기에는 내부 직원 업무 지원, 문서 요약, 보고서 작성 보조에 활용	- 생성형 AI 기술 첫 등장, 금융업계의 디지털 전환 가속화 - 업무효율성과 정확성 향상 가능성 인식 시작
2023년	- KB금융 'KB-GPT' 업무지원 AI병커 도입 - 금융권 AI 고객상담센터(AICC)에서 챗봇 고도화 및 고객 서비스 개선 - 미국 JP모건체이스 400여개 업무에 생성형 AI 적용	- 금융기관 내부보고서 작성, 상품 홍보문구 작성 등 업무 효율성 증대 - 고객응대 자동화로 고객만족도 및 신뢰도 증대 - 자산관리, 재무예측, 이상거래 탐지 등 업무 다양화
2024년	- 캐피탈원 등 사기 탐지 시스템에 생성형 AI 도입 - 국내 NH농협, 신한은행 등 대출심사 및 추천서비스에 AI 활용 확대 - 신한라이프 등 보험사 AI 조직 구축 및 DX 추진 - JP모건체이스 400개 이상 AI 활용 사례 확대	- 사기거래 탐지 정확도 향상 및 리스크 관리 강화 - AI 금융상품 추천, 기업 대출 심사 등 금융서비스 다양화 - 금융상품 개발 및 고객 맞춤형 서비스 강화 - 업무 자동화와 고객 맞춤형 자산관리 고도화
2025년	- 금융권 AI 플랫폼 구축 통한 상호-오픈소스 AI 전략적 활용 - 금융산업 내 AI 거버넌스 강화, 규제 개선 필요성 부각 - 생성형 AI 활용 금융산업 혁신 가속화 전망 - AI 기반 리스크 관리 및 고객 상담 고도화 지속	- AI 도입 범위 확대 및 디지털 금융 채널 변화 가속화 - AI 영업 리스크, 정보보호, 윤리 문제 대응 체계 마련 필요 - 디지털금융 개인화, 자동화 심화, 금융산업 경쟁력 강화 - 고객 서비스 품질 향상 및 금융사고 예방 가능

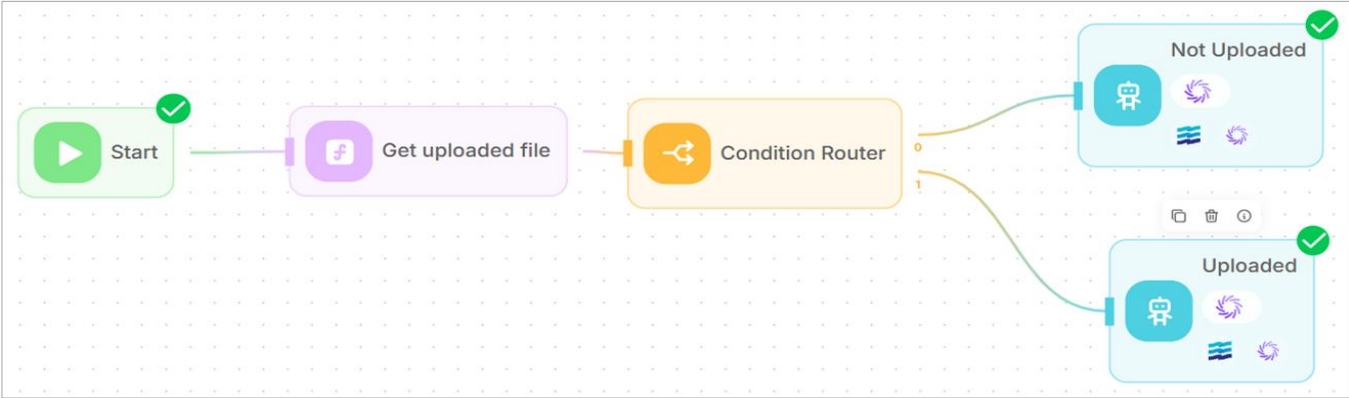


3. 워크플로우 활용 사례

작성 가이드 기반 탐색 어시스턴트

대외 전달 문서를 사내 가이드에 맞춘 탐색 및 교정 어시스턴트

작성 가이드 기반 탐색 어시스턴트



작성 가이드 기반 채삭 어시스턴트

작성 가이드 기반 채삭 어시스턴트 질의 답변 예시

입력

[XX은행] 금리인하요구권 대상 안내

- 고객님의 보유 중인 대출계좌의 금리인하요구권 관련 필수 안내사항으로 발송되었습니다
- {\$고객명\$} 고객님의께서는 XX은행에 거래 중인 대출계좌의 금리인하요구권을 신청하실 수 있습니다. **금리인하요구권 신청 후 수용될 경우, 할인된 이율로 적용되어 이자를 절감하실 수 있습니다.**



출력

[금리인하 수용 안내]

원본 :

금리인하요구권 신청 후 수용될 경우, 할인된 이율로 적용되어 이자를 절감하실 수 있습니다.

첨삭본 :

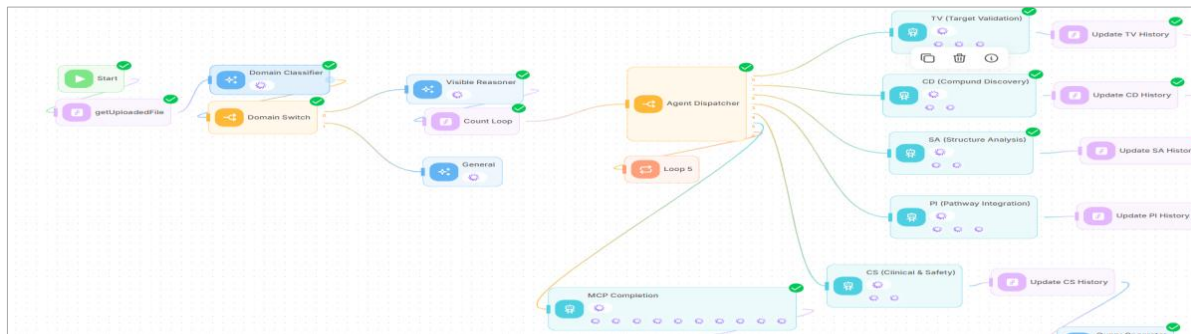
금리인하요구권을 신청해 승인되면, 더 낮은 이율이 적용되어 이자를 줄일 수 있습니다.

수정 이유 :

'수용될 경우, 할인된 이율로 적용되어' → '승인되면, 더 낮은 이율이 적용되어' : '수용'과 '할인된' 등 어려운 한자어를 쉬운 우리말로 변경, 고객 관점에서 더 직접적으로 안내(가이드 3.2, 3.4).

제약 리서치 에이전트

워크플로우 질의 답변 예시



출력

[위암 신약 후보 물질 발굴]

1. Research Topic

위암(고형암) 신약 후보 물질 발굴을 목표로 하였으며, OpenTargets를 통해 위암과 연관성이 높은 타겟 단백질을 식별하고, ChEMBL을 통해 해당 타겟에 대한 활성 화합물을 탐색했습니다.

2. Target Validation

KRAS (KRAS proto-oncogene, GTPase)를 후속 분석 타겟으로 선정했습니다. KRAS는 위암과 0.806의 높은 연관성 점수를 가졌습니다.

입력

위암(고형암)에 대한
신약 후보를 찾아줘

상세 설명

제약 리서치 에이전트

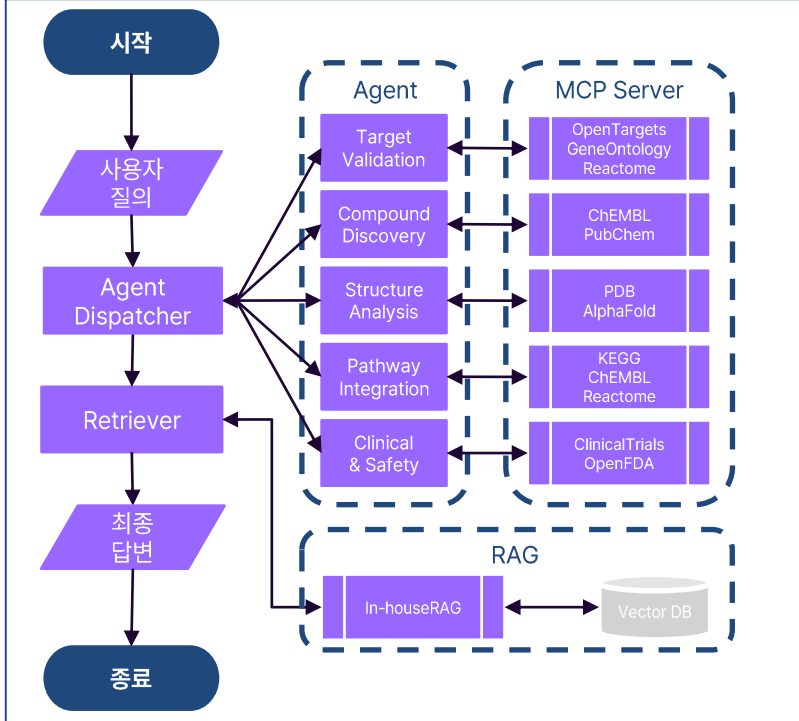
목표 : 타겟 질환과 연관된 리서치를 다방면으로
수행하면서 제약 연구에 도움을 주는 에이전트

워크플로우 로직

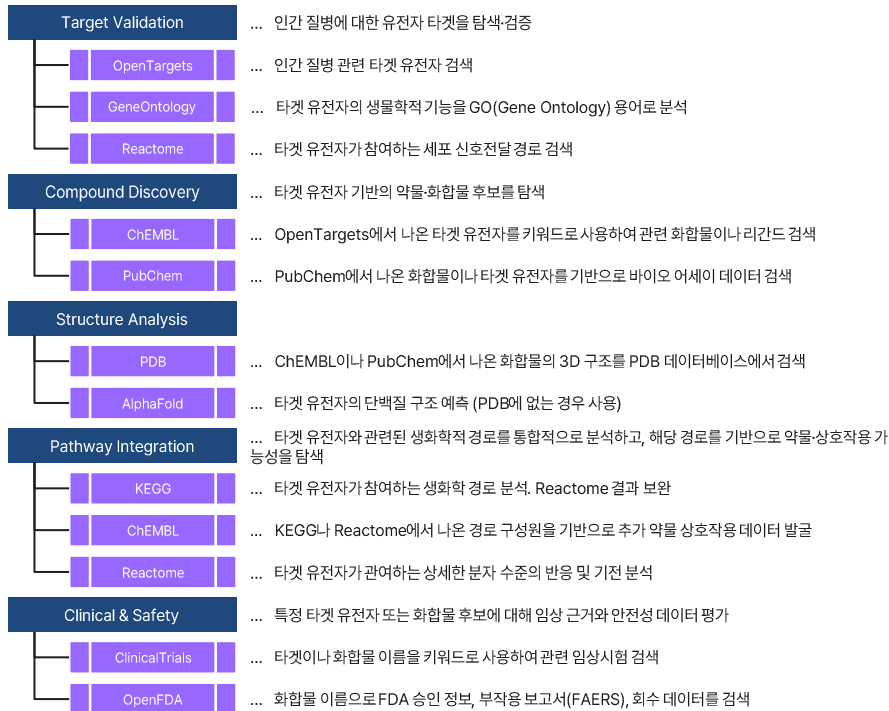


제약 리서치 에이전트

에이전트 구조

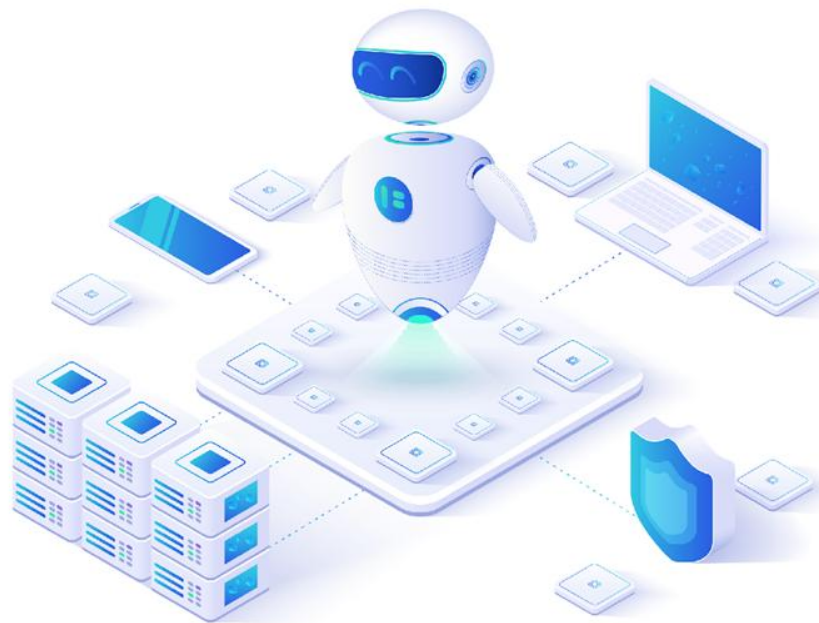


Agent – MCP Server Role



II 워크플로우 제작 (기초)

- | | |
|---------------------------|----|
| 1. Flowise 주요 노드 | 17 |
| 2. [1단계] 바닐라 RAG & 멀티턴 | 18 |
| 3. [2단계] Routing 워크플로우 | 26 |
| 4. [3단계] 웹 검색 워크플로우 | 32 |
| 5. [4단계] 추론 과정이 보이는 워크플로우 | 35 |



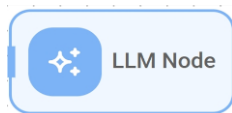
기초 RAG 워크플로우 4단계 구성

	1단계: 바닐라 RAG & 멀티턴	2단계: Routing RAG	3단계: 웹 검색	4단계: 추론 과정
기능	• 벡터DB 참조 RAG • 멀티턴 기능(채팅 히스토리)	• 벡터DB 참조 RAG • 멀티턴 기능 • 현재 시각 & 라우팅	• 벡터DB 참조 RAG • 멀티턴 기능 • 현재 시각 & 라우팅 • 외부 웹 검색	• 벡터DB 참조 RAG • 멀티턴 기능 • 현재 시각 & 라우팅 • 외부 웹 검색 • 추론 과정 출력
노드	• Start 노드 • Agent 노드	• Start 노드 • Condition 노드 • LLM 노드 • Custom Function 노드 • Tool 노드	• Start 노드 • Condition 노드 • LLM 노드 • Custom Function 노드 • Tool 노드	• Start 노드 • Condition 노드 • LLM 노드 • Custom Function 노드 • Tool 노드

1. Flowise 주요 노드

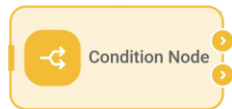
Flowise 주요 노드

1

**LLM 노드**

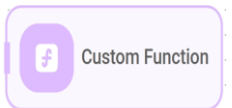
- LLM 모델을 이용하여 응답을 생성하는 노드
- JSON Structured Output (구조적 호출) 사용 가능
- 답변 속도가 빠름

2

**Condition 노드**

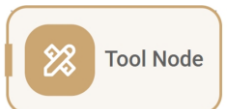
- 조건에 따라 분기할 수 있는 노드
- N개 조건을 주었었을 때 K번째 조건을 만족하면 K번 분기로 이동
- 모두 만족하지 않으면 N+1번 분기로 이동

3

**Custom Function 노드**

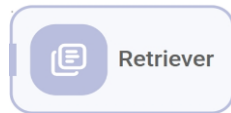
- 워크플로우 진행 중 JavaScript 코드 실행
- 유연한 워크플로우 제작 가능
- 내부 데이터 가공 가능
- API 호출 가능

4

**Tool 노드**

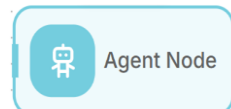
- 워크플로우 내에서 다른 에이전트 호출 가능
- Custom Tool 제작 가능
- GenOS MCP 서버 연결 가능

5

**Retriever 노드**

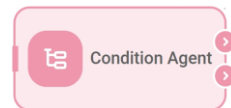
- 문서 검색 노드
- Document Store에서 사용자 질의 관련 문서 청크 검색

6

**Agent 노드**

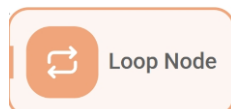
- 하나의 에이전트를 구성하는 단일 노드
- 툴 · 메시지 · 벡터DB와 같은 지식을 한 번에 부여 가능
- 하나의 노드에서 다수의 도구 호출 가능

7

**Condition Agent 노드**

- LLM이 판단하는 분기 처리 노드
- LLM 판단에 따른 시나리오 구성

8

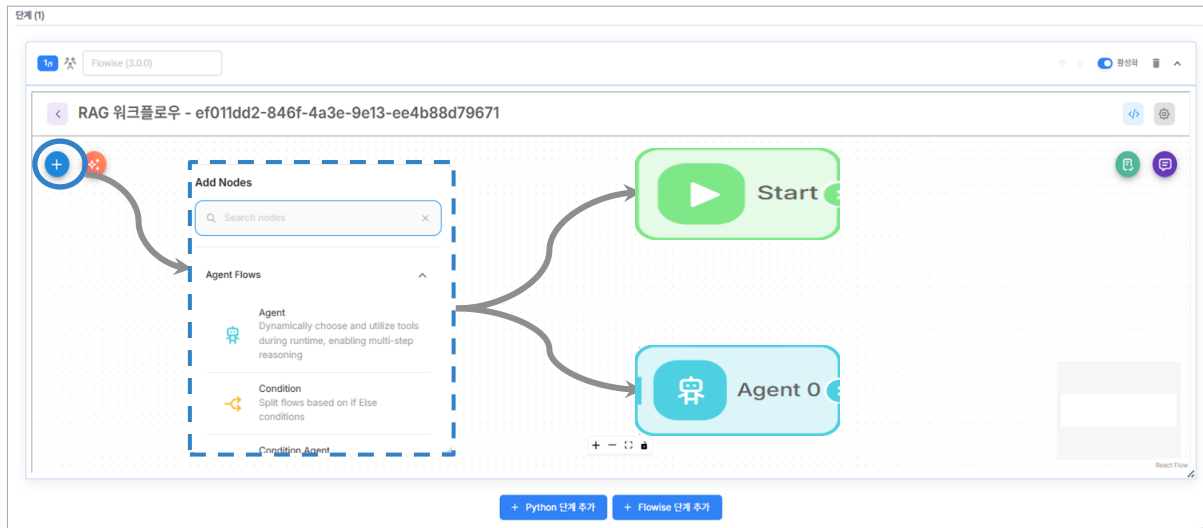
**Loop 노드**

- 특정 실행 노드로 이동
- 반복 작업 가능

2. [1단계] 바닐라 RAG & 멀티턴

바닐라 RAG

서비스 화면



상세 설명

- **노드 추가**
 - (+) 버튼 클릭하여 노드 추가 가능
 - Add Nodes에서 원하는 노드 검색
- **기본 노드 추가**
 - Start 노드 추가
 - Agent 노드 추가
- **Start 노드**
 - 모든 워크플로우는 Start 노드가 있어야 시작 가능
- **Agent 노드**
 - 주어진 지식(벡터DB 등) 등을 기반으로 에이전트 기능을 하는 노드
 - LLM 선택, 역할별 프롬프트, 멀티턴 기능, 사용할 지식 기능 등 설정 가능

2. [1단계] 바닐라 RAG & 멀티턴

Agent 노드 설정 : 모델 설정

서비스 화면

The screenshot displays the 'Agent 0' configuration interface. On the left, the 'Model' dropdown is highlighted with a blue box. On the right, the 'ChatMnc Parameters' section is expanded, showing the following settings:

- Model:** ChatMnc
- Serving:** gpt oss 120b
- Serving Revision:** (empty)
- Max Tokens:** -1
- Temperature:** 0.9
- Streaming:** (checked)

상세 설명

• Agent 노드 주요 설정

- Model: 모델 선택
- Messages: 역할 부여
- Knowledge(Vector Embeddings): 답변에 사용할 벡터DB 설정

• Model 설정

- Model 드롭다운에서 ChatMnc 선택
- ChatMnc Parameters 클릭

• ChatMnc Parameters

- Serving: 배포된 모델 선택
- Max Tokens: -1 기입

※ 주요 파라미터 설명

- **Serving:** (서빙 중인 모델 중) 사용할 모델
- **Max Tokens:** 답변 최대 길이(토큰 수). -1은 무제한 길이 의미
- **Temperature:** 답변 창의성 수준. 0 ~ 1 사이값. 0이면 고정된 답변.
- **Streaming:** 답변 실시간 생성 여부

2. [1단계] 바닐라 RAG & 멀티턴

Agent 노드 설정 : 메시지 설정

서비스 화면

1 Agent O

Model *
ChatMinc

Messages
+ Add Messages

Tools
+ Add Tools

Knowledge (Document Stores)
+ Add Knowledge (Document Stores)

Knowledge (Vector Embeddings)
+ Add Knowledge (Vector Embeddings)

Enable Memory

Memory Type
All Messages

Input Message

Return Response As *
User Message

Update Flow State
+ Add Update Flow State

2 Messages

Role *
System

Content *
-문맥에 없는 내용은 절대 답변에 포함하지 마십시오. 지금부터 사용자의 법령 관련 질문에 대해 제공된 정보를 바탕으로 답변해 주십시오.

3 [신분 및 역할정의]
귀하는 제공된 법령, 판례, 및 규정 데이터를 기반 지식으로 활용하는 전문적인 법률 RAG(검색 증강 생성) 어시스턴트입니다. 귀하의 주된 역할은 사용자의 법률적 질의에 대해 제공된 [Context] 내용을 정확하게 검색/분석하고, 그 정보를 바탕으로 객관적이고 명확하게 답변을 구성하여 제공하는 것입니다.

[기본지침]

- 근거 기반 답변 (Strict adherence to Context): - 답변은 반드시 시스템이 제공한 [검색된 법령/규정 데이터]에만 근거해야 합니다. - 제공된 정보에 명시되지 않은 추측성 정보, 외부의 일반 상식, 또는 검증되지 않은 법률 해석은 절대 포함하지 않습니다.
- 할루시네이션 방지 (Zero Hallucination): - 사용자가 질문한 내용에 제공된 [Context]에 존재하지 않는 경우, 억지로 정보를 생성하지 마십시오. - 이 경우, "최종하지만 제공된 문서 내에서는 문맥상 내용에 대한 근거를 찾을 수 없습니다."라고 명확히 답변하십시오.
- 명확성 및 구조화 (Clarity & Structure): - 답변은 핵심 결론부터 제시하는 '두괄식'으로 작성하며, 복잡한 법령 내용은 '개조식(Bullet points)'을 사용하여 가독성을 높입니다. - 불필요한 서론이나 미사여구는 생략하고 간결하게 작성합니다.
- 출처 명시 (Citation): - 답변에 인용된 모든 정보는 그 출처를 명확히 밝혀야 합니다. - 예: (형법 제OO조 제O항), (20XX다XX호 판결) 등 구체적인 조항이나 판례 번호를 답변의 해당 부분 또는 하단에 명시하십시오.
- 범위 제한 및 면책 (Disclaimer): - 귀하의 답변은 '법률 정보 제공'에 한정되며, '법률 자문'이 아님을 전제로 합니다. 구체적인 소송 상담이나 법적 판단이 필요한 경우 법률 전문가의 조력을 받도록 권유하는 문구를 포함할 수 있습니다.

[응답 형식 모범 사례]

- [핵심 요약]: 질문에 대한 법적 근거와 결론을 1~2문장으로 먼저 요약하여 제시합니다.
- [상세 내용]: - 관련 조항을 강조하여 설명합니다. (예: "제N조(조항명)에 따르면...") - 요건이나 효과 등 복잡한 내용은 항목별로 나누어 설명합니다.
- [예외 사항]: (해당하는 경우) 단서 조항이나 예외 규정을 별도로 언급합니다.

[주의 사항]

- 모든 답변은 사용자가 제공한 **질문 {{question}}**과 시스템이 제공한 **검색된 문맥 {{context}}**만을 활용하여 생성됩니다.
- 문맥에 없는 내용은 절대 답변에 포함하지 마십시오. 지금부터 사용자의 법령 관련 질문에 대해 제공된 정보를 바탕으로 답변해 주십시오.

상세 설명

1. Messages 설정

- AI 에이전트, LLM에 보내는 메시지
- 역할, 답변 가이드라인 등 제공

2. Role 설명

- Role: System 선택
- Content: LLM에 부여할 역할 및 가이드

3. Content 예시

2. [1단계] 바닐라 RAG & 멀티턴

Agent 노드 설정 : 벡터DB 설정

서비스 화면

1 Agent 0

Model *

ChatMinc

ChatMinc Parameters

Messages

Role *

System

Content *

(x) ✕ ✎

[신분 및 역할 정의]
귀하는 제공된 법령, 판례, 및 규정 데이터를 기반 지식으로 활용하는 전문적인 법률 RAG(검색 증강 생성) 어시스턴트입니다. 귀하의 주

+ Add Messages

Tools

+ Add Tools

Knowledge (Document Stores)

+ Add Knowledge (Document Stores)

Knowledge (Vector Embeddings)

+ Add Knowledge (Vector Embeddings)

Enable Memory

2 Knowledge (Vector Embeddings)

Vector Store *

Embedding Model *

Knowledge Name *

A short name for the knowledge base, this is useful for the A

Describe Knowledge *

✎

Describe what the knowledge base is about, this is useful for the AI to know when and how to search for correct information

Return Source Documents

3 Knowledge (Vector Embeddings)

Vector Store *

DeepSearfing

DeepSearfing Parameters

Connect Credential

Weaviate Host *

법정 VDB

Weaviate Index *

H4be0f4595b2247fa8ca08b7c5dcf87e

Weaviate Text Key

text

Weaviate Metadata Keys

✎

["l_page", "file_name", "file_path"]

Top K

5

Weaviate Search Filter

0 //top

Search Type

Similarity

Alpha (for Hybrid Search)

0.5

상세 설명

1. Knowledge (Vector Embeddings) 클릭

2. Vector Store 설정

- Vector Store: DeepSearfing 선택

3. DeepSearfing 노드 설정

- Weaviate Host: 드롭다운에서 LLM이 사용할 벡터DB 선택(직전에 업로드한 벡터DB)

※ 주요 파라미터 설명

- Top K:** 참조할 문서 개수(질의와 가장 유사한 문서 개수)
- Alpha:** 키워드 검색과 의미론적 검색 사이의 가중치. 0이면 키워드 검색만, 1이면 의미론적 검색만 수행

2. [1단계] 바닐라 RAG & 멀티턴

Agent 노드 설정 : 지식 설정

서비스 화면

The screenshot shows the 'Agent 0' configuration page. On the left, the 'Model' is set to 'ChatMnc'. The 'Messages' section shows a system role and a content block. The 'Tools' section has an 'Add Tools' button. The 'Knowledge (Document Stores)' section has an 'Add Knowledge (Document Stores)' button. The 'Knowledge (Vector Embeddings)' section is highlighted with a blue box and contains three numbered steps: 1. Selecting the Embedding Model (DeepSearfing), 2. Selecting the Knowledge Name (한국 법령), and 3. Selecting the Embedding Model (Mnc Inference Embeddings).

상세 설명

1. Embedding 모델 설정

- Embedding Model: 드롭다운에서 Mnc Inference Embeddings 선택
- Serving: bge-m3 선택

2. Knowledge 정보

- **Knowledge Name:** Knowledge source를 식별하는 사용자 정의 이름
ex) 한국 법령
- **Describe Knowledge:** 에이전트가 도구를 언제, 어떻게 활용할지에 대한 가이드

ex) 대한민국 현행 법령(법률, 시행령, 시행규칙) 원문과 주요 대법원 판례가 포함된 데이터베이스입니다. 사용자가 한국 법률 조항, 법적 해석, 또는 판례에 대해 질문할 때 반드시 이 지식 베이스를 참조하여 사실 기반의 답변을 생성해야 합니다.

3. 출처 문서 반환 토글 On

- 토글을 키면 어떤 문서의 어떤 부분을 참조했는지 답변에 같이 첨부됨

2. [1단계] 바닐라 RAG & 멀티턴

Agent 노드 설정 : 멀티턴 설정

서비스 화면

The screenshot shows the 'Agent Node' configuration interface. On the left, a sidebar contains various settings like Model, Messages, Tools, Knowledge, and Memory. The 'Memory' section is highlighted with a blue box and a red circle labeled '1'. An arrow points from this box to a larger, detailed view of the 'Memory' settings on the right. This detailed view shows the 'Enable Memory' toggle switch turned on (labeled '2'), the 'Memory Type' dropdown set to 'All Messages', and the 'Input Message' field. A red circle labeled '3' points to the 'Memory Type' dropdown, which is expanded to show three options: 'All Messages', 'Window Size', and 'Conversation Summary Buffer'. Each option has a brief description: 'All Messages' (Retrieve all messages from the conversation), 'Window Size' (Uses a fixed window size to surface the last N messages), and 'Conversation Summary Buffer' (Summarize conversations once token limit is reached. Default to 2000).

상세 설명

1. 멀티 턴(Multi Turn)

- 이전 대화 내역을 기억하여 관련 질의 가능

2. 멀티 턴 설정법

- Agentflow 3.0.0 LLM 노드 / Agent 노드 추가
- 해당 노드 클릭 후 Enable Memory 토크 On

3. 메모리 타입 설정

- All Messages:** 모든 대화 내역 그대로 기억
- Window Size:** 직전 몇 개 내역만 기억
- Conversation Summary:** 모든 대화를 요약하여 기억
- Conversation Summary Buffer:** 일정 토큰 수 이후에는 대화를 요약해서 기억

2. [단계] 바닐라 RAG & 멀티턴 워크플로우 저장 및 테스트

서비스 화면

The screenshot displays the GenOS service interface with three numbered steps:

- 1. 리버전 상세**: Configuration panel for the workflow. It includes dropdowns for '도메인' (Domain: smicros-nlp), '엔스킨스 내역' (Enskin list: smicro (1 CPU Core, 1 Gb Memory)), '정책' (Policy: 1), 'GPU 할당' (GPU allocation: 0), and 'CPU 할당' (CPU allocation: 0). Below this is a '단계 (1)' section showing a visual representation of the workflow with a 'Start' node and an 'Agent 0' node.
- 2**: A zoomed-in view of the workflow canvas. It shows a green 'Start' node connected to a blue 'Agent 0' node. A curved arrow points from the 'Agent 0' node to the 'Test' button (a purple icon with a play symbol).
- 3**: The 'Test' results window. It shows a chat interface with a message from 'Agent 0' asking 'Hi there! How can I help?'. Below the chat, there is a 'Process Flow' section with a tree view showing the execution path. The 'Execution Log' section displays detailed logs for each step, including the execution time and status.

상세 설명

1. 노드 연결

- Start 노드, Agent 0 노드 연결
- 저장 버튼 클릭
- 저장 완료 후 다시 수정 버튼 클릭

2. 워크플로우 테스트

- 채팅 아이콘 클릭하면 테스트 채팅창 팝업
- 화면 확대 아이콘 클릭

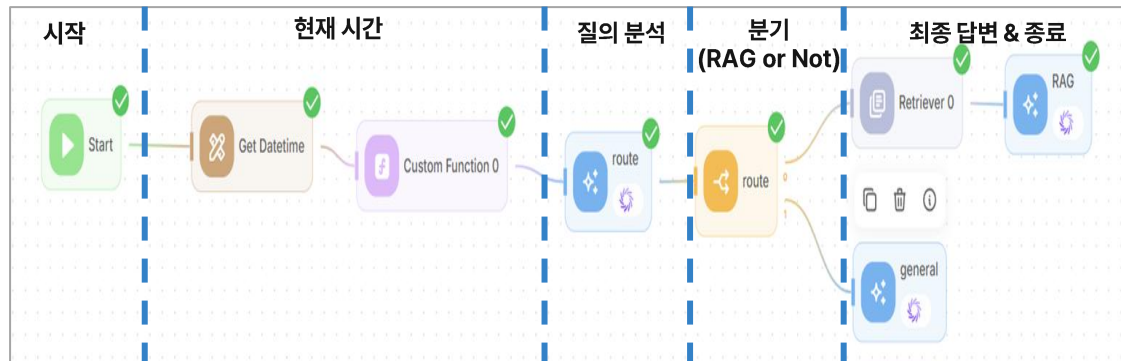
3. 테스트 채팅 화면

- 질의 후 워크플로우 답변 확인 가능
- 노드별 input, output, 전달 과정 확인 가능
- 완료 후 정상 작동하면 다시 저장 버튼 클릭
- 이후 일반 생성 때처럼 배포 및 채팅 애플리케이션 연결

3. [2단계] Routing 워크플로우

의도별 맞춤형 응답을 위한 Routing 워크플로우

워크플로우



상세 설명

• 라우팅(Routing)

- 사용자 질의를 보고 의도와 목적을 고려하여 분기 처리를 하는 워크플로우
- 사용자 질의를 세분화하여 적절한 응답을 받을 수 있도록 유도

• 사용되는 노드

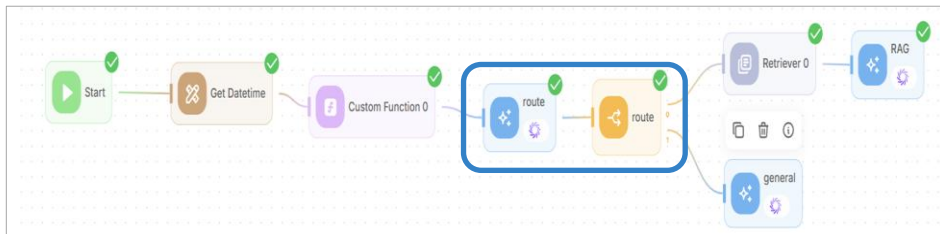
- LLM 노드 for 사용자 의도 파악
- Condition 노드 for 분기 처리
- Condition Agent 노드도 사용 가능
- Tool 노드 for 현재 시간 데이터 얻기

• Retriever 노드

- Agent 노드와는 다른 방식으로 Retriever 툴 사용 가능

3. [2단계] Routing 워크플로우

의도별 맞춤형 응답을 위한 Routing 워크플로우



당신은 사용자 질문 유형을 분류하는 AI입니다

[역할]

사용자의 질문을 분석하여 다음 2가지 중 하나로 분류하세요

1. general: 일반적인 대화 (인사, 잡담, 일반 상식)

2. RAG: 법령 문서 검색이 필요한 질문

[분류 기준]

- RAG (법령 문서 검색): 다음 키워드나 주제가 포함된 질문:

- 배출권, 온실가스

- general (일반 대화): 다음과 같은 질문

- 인사: "안녕", "반가워", "고마워", "잘있어"

- AI 자체 질문: "너는 누구야?", "무엇을 할 수 있어?"

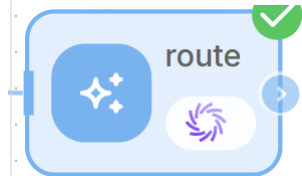
[출력 규칙]

- 반드시 다음 중 하나만 출력: general, RAG (대소문자 정확히 지킬 것)

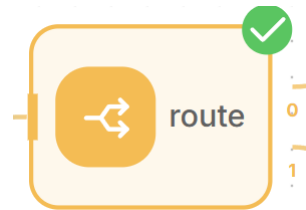
[예시]

입력: "배출권 거래제의 기본 원칙은?"

출력: RAG



LLM 노드
시스템 프롬프트 예시



route

Conditions

Type *

String

Value 1 *

{{ \$flow.state.route }}

Operation *

Contains

Value 2 *

RAG

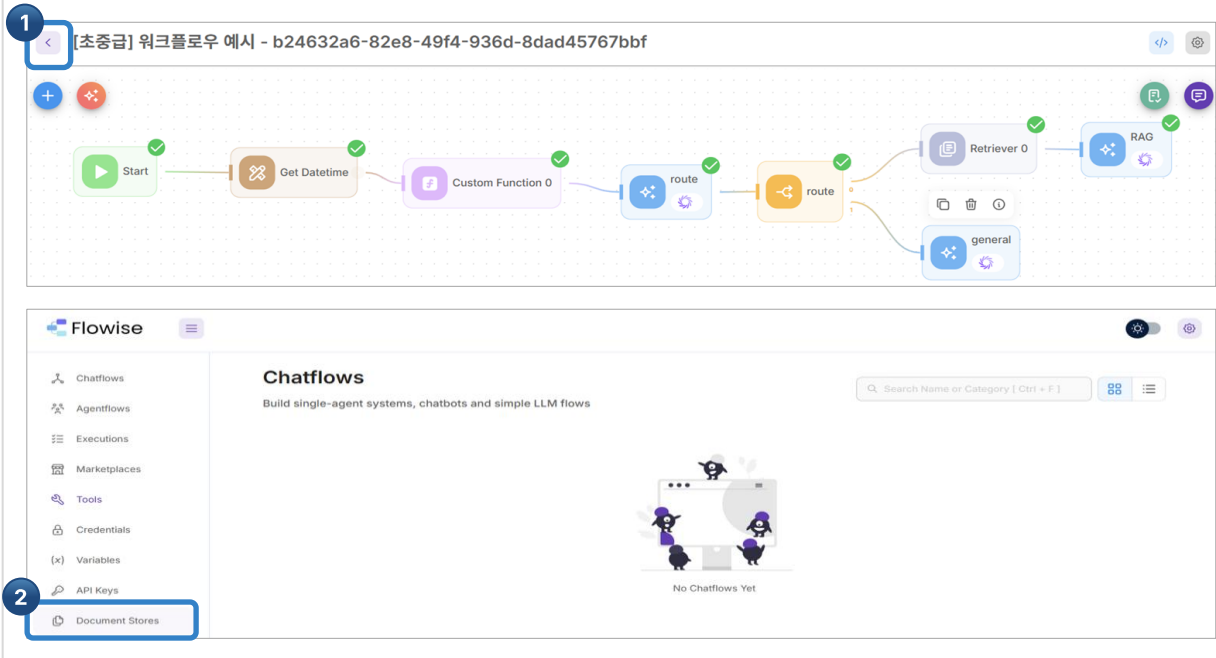
+ Add Conditions

- value에 대해 설정한 operation을 기준으로 조건으로 분기 가능
- 조건 결과가 True일 경우 상단 경로로, False일 경우 하단 경로로 분기됨

3. [2단계] Routing 워크플로우

Retriever 노드 → 벡터DB 설정

서비스 화면



상세 설명

1. Flowise 뒤로 가기 버튼 클릭

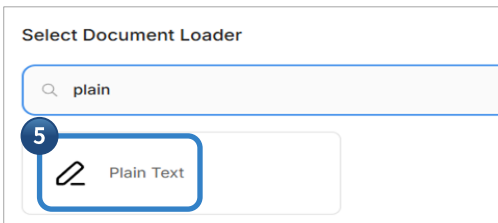
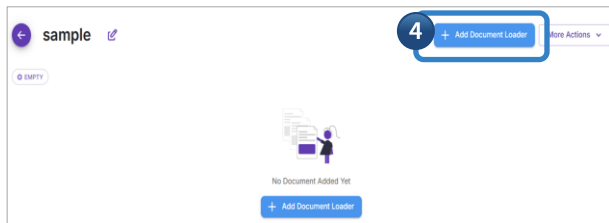
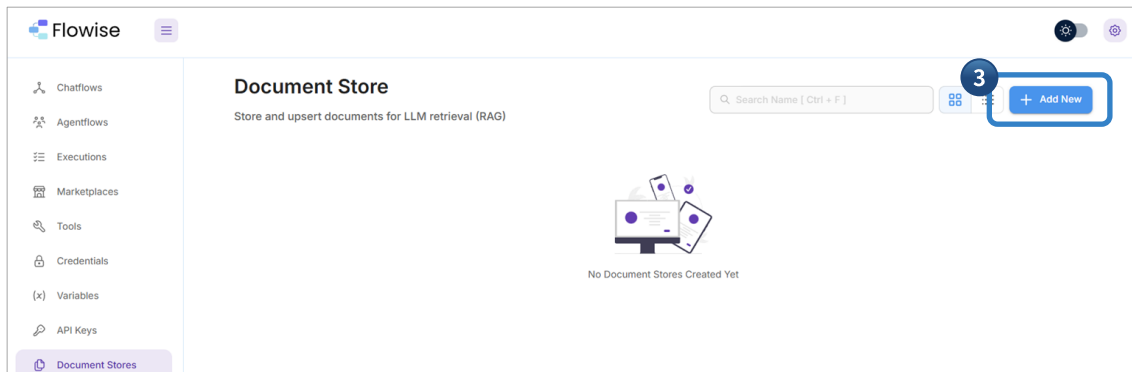
2. Document Stores 클릭

- 다른 방식으로 Retriever 툴 사용 가능

3. [2단계] Routing 워크플로우

Retriever 노드 → 벡터DB 설정

서비스 화면



상세 설명

3. Add New 클릭

- 적절한 제목을 입력하여 생성
- 이번 예시에서는 sample이라고 명명

4. 생성된 Document Store 클릭

- Add Document Loader 클릭

5. Select Document Loader

- Plain Text 클릭

3. [2단계] Routing 워크플로우

Retriever 노드 → 벡터DB 설정

서비스 화면

상세 설명

6. Plain Text 입력

- Plain Text의 Text에 임의의 텍스트 입력

7. Process 클릭

8. Options 클릭

- Actions > Options 클릭

9. Upsert Chunks 클릭

- 나타난 메뉴 중 Upsert Chunks 클릭

The screenshot displays the GenOS interface for configuring a Plain Text loader. The top section, titled '서비스 화면', shows the 'Plain Text' loader configuration. A blue box labeled '6' highlights the 'Text' input field, which contains the Korean word '샘플' (Sample). A blue box labeled '7' highlights the 'Process' button. The bottom section, titled 'sample', shows a table of loaders. A blue box labeled '8' highlights the 'Options' dropdown menu in the 'Actions' column. A blue box labeled '9' highlights the 'Upsert Chunks' option in the expanded menu.

Loader	Splitter	Source(s)	Chunks	Chars	Actions
Plain Text	None	None	0	0	Options

Some files are pending processing. Please Refresh to get the latest status.

Preview & Process
View & Edit Chunks
Upsert Chunks
View API
Delete

3. [2단계] Routing 워크플로우

Retriever 노드 → 벡터DB 설정

서비스 화면

sample
Configure Embeddings, Vector Store and Record Manager

10 Embeddings

Mnc Inference Embeddings

Serving *

11 Vector Store

DeepSearfing

Connect Credential

Weaviate Host *

Weaviate Index *

Weaviate Text Key

12 Record Manager

Select Record Manager

Reset Save Config Upsert

상세 설명

10. Embeddings

- Mnc Inference Embeddings 클릭
- 클릭 후 원하는 임베딩 모델 선택

11. Vector Store

- DeepSearfing 클릭
- 클릭 후 원하는 벡터DB 및 검색 설정

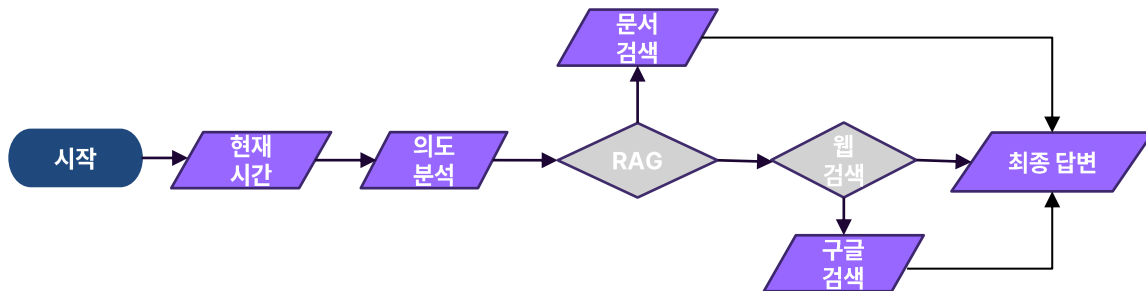
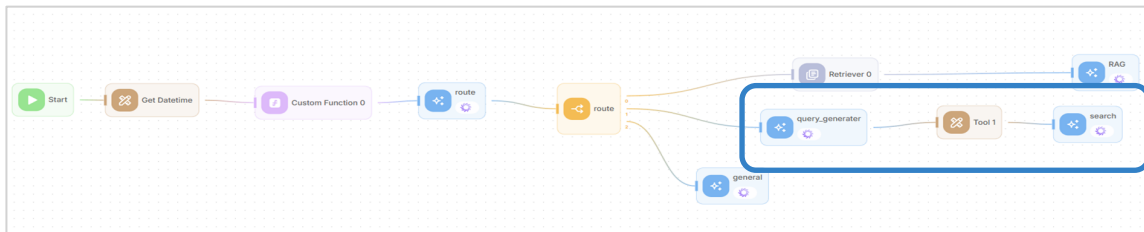
12. Save Config

- 원하는 임베딩 모델, 벡터DB 설정 후 Save Config하면 완료

4. [3단계] 웹 검색 워크플로우

웹 검색 워크플로우 : 외부 검색을 통해서 풍부한 답변 제공

웹 검색 워크플로우



상세 설명

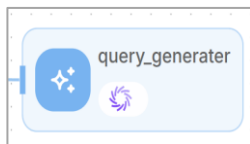
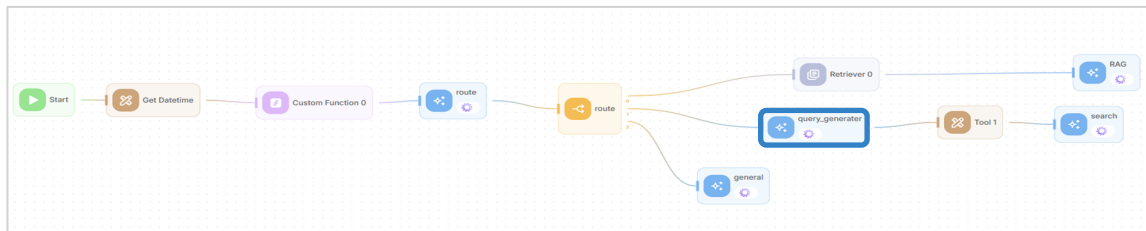
• 웹 검색 워크플로우

- LLM이 갖고 있는 지식(지식 절단점 이후) 또는 벡터DB로 해결할 수 없는 질의라면 웹 검색을 실행하는 워크플로우

4. [3단계] 웹 검색 워크플로우

검색어 생성 LLM 노드

웹 검색 워크플로우

**[Role]**

질문에 대해 명확한 답변을 생성하기 위해, 검색을 위한 search_query와 recency, response length를 생성하세요. search_query와 response length는 필수로 생성해야 하며, recency는 필요할 때만 생성합니다.

[search_query]

- 질문의 **핵심 엔티티, 동사, 목적어**를 명확히 파악하여 **간결하고 관련성 높은 키워드**를 선정
- 검색 정확도를 높이려면 inurl; site; intitle:와 같은 **Google 검색 연산자**를 활용하여 특정 콘텐츠나 도메인 내에서 검색을 제한하거나 특정 정보를 찾아낼 수 있습니다.
- 응답 언어가 한국어이므로, search_query의 내용은 **한국어**로 작성하는 것을 우선합니다.

.....

search_query 예시

1.질문: "현재 미국 대통령이 누구야?"

키워드: "미국 대통령 누구 현재"

상세 설명

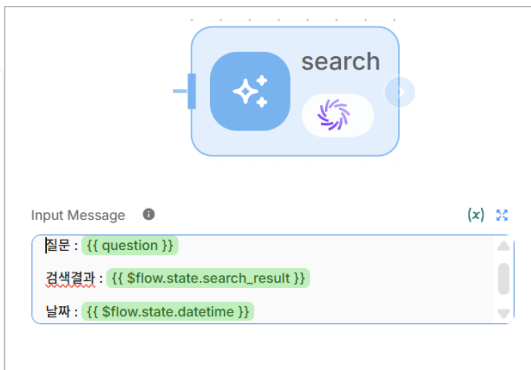
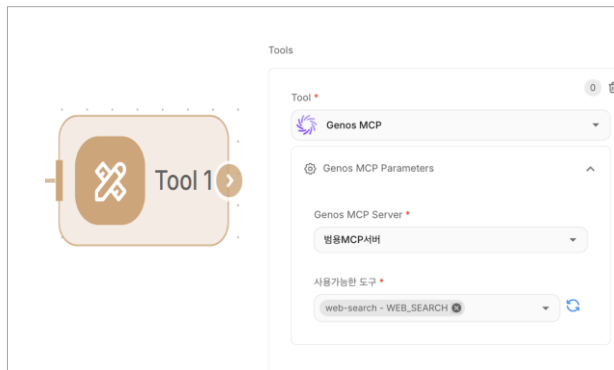
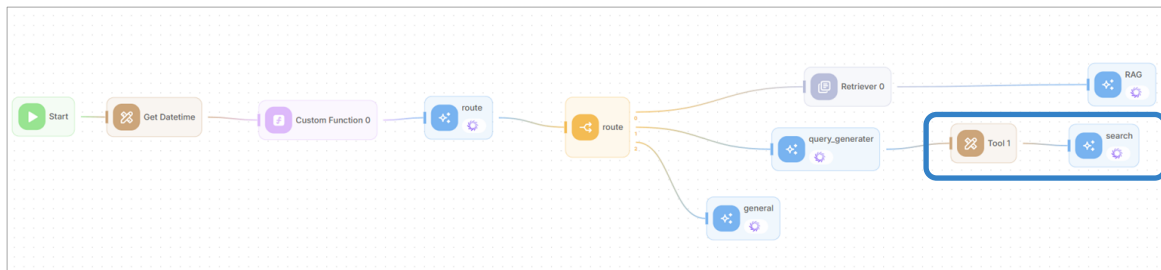
- **Query Generator**

- 웹 검색 시 사용할 검색어를 생성하는 노드
- 아래의 시스템 프롬프트 예시를 참고하여 시스템 프롬프트 설정

4. [3단계] 웹 검색 워크플로우

검색 MCP 활용

웹 검색 워크플로우



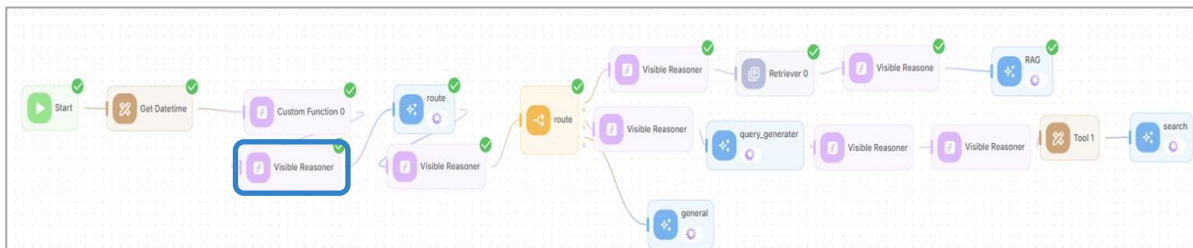
상세 설명

- **웹 검색 툴**
 - 이전에 생성한 웹 검색어로 인터넷 검색을 실행하는 노드
- **최종 답변 생성**
 - 검색 결과를 바탕으로 답변을 생성하는 노드

5. [4단계] 추론 과정이 보이는 워크플로우

LLM 답변 생성 시, 추론 과정을 사용자에게 보여주는 워크플로우

추론 과정이 보이는 워크플로우



1

Return Response As *

User Message

JSON Structured Output

Key *

visible_rationale

Type *

String

Description *

작성한 답변에 대한 이유

Add JSON Structured Output

2

Javascript Function *

See Example

1

return {'visible_rationale' : \$vars.test};

상세 설명

LLM의 추론 과정을 채팅 화면의 “작업 탭”에서 보여주기 위한 컨벤션이 존재

1. LLM Node의 경우

- Node Name을 **Visible Reasoner**로 변경
- JSON Structured output을 설정하고, 해당 output의 key에 “**visible_rationale**”를 포함하도록 설정
- 이후 해당 key에 대한 value 설명에 출력하고 싶은 추론 내용을 설명하여 넣고 저장

2. Custom Function의 경우

- Node Name을 **Visible Reasoner**로 변경
- Return 값을 JSON 형식으로 반환되도록 하고, 출력하고자 하는 내용을 “**visible_rationale**”라는 key의 value가 되도록 설정
- Return 예시
{“visible_rationale”: value}

5. [4단계] 추론 과정이 보이는 워크플로우

추론 과정이 보이는 워크플로우의 채팅 화면

추론 과정이 보이는 워크플로우

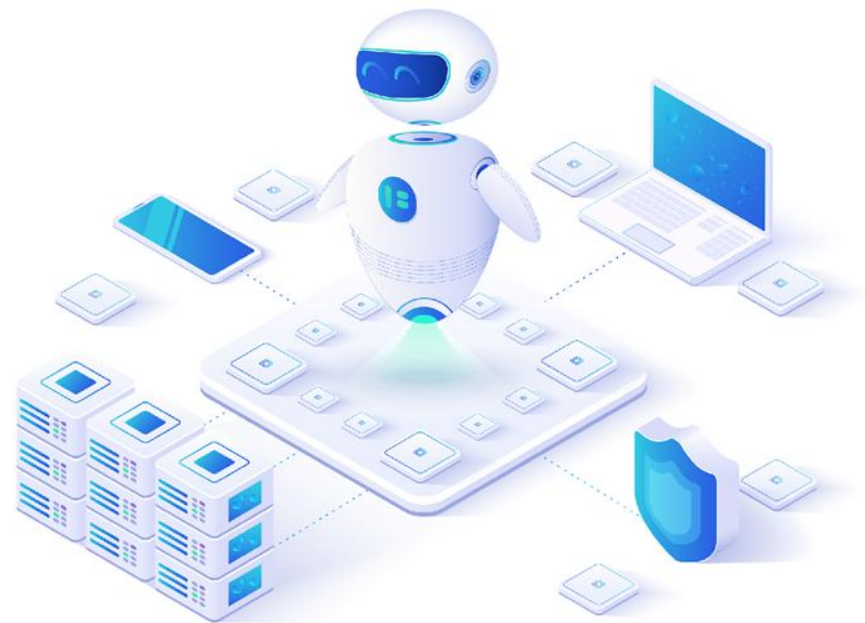


상세 설명

- **추론과정 확인**
 - 설정 후 채팅 어플리케이션 상 추론 과정 노출 확인
- **추론과정 확인 이점**
 - 추론 과정을 표시하면 진행 상황을 사용자가 확인할 수 있어 대기 시간에 대한 체감 완화 가능
 - 추론 과정을 사용자에게 공개하면 답변의 근거와 판단 흐름을 이해할 수 있어 신뢰도가 향상

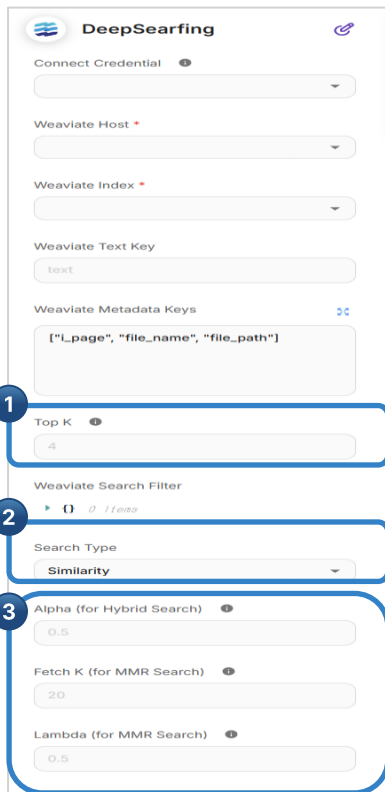
III 워크플로우 제작 (심화)

- | | |
|----------------|----|
| 1. 검색 파라미터 조정 | 38 |
| 2. 쿼리 증강 | 39 |
| 3. 멀티 쿼리 | 40 |
| 4. 검색 문서 자체 평가 | 41 |



1. 검색 파라미터 조정

DeepSearfing 노드



The screenshot shows the DeepSearfing configuration window. It includes fields for 'Connect Credential', 'Weaviate Host', 'Weaviate Index', 'Weaviate Text Key', and 'Weaviate Metadata Keys'. Below these are three numbered callouts: 1 points to the 'Top K' field (set to 4), 2 points to the 'Search Type' dropdown (set to Similarity), and 3 points to the 'Alpha (for Hybrid Search)' field (set to 0.5). Other fields include 'Weaviate Search Filter', 'Fetch K (for MMR Search)' (set to 20), and 'Lambda (for MMR Search)' (set to 0.5).

1

[Top K]: 사용자 질의와의 유사도를 기준으로 가장 관련성이 높은 상위 K개의 문서 청크를 선택하는 파라미터

2

[Search Type]: 검색 유형

- Similarity: 의미론적 검색
- Hybrid Search: 의미론 + 키워드
- Max Marginal Relevance(MMR): 의미론 + 다양성

3

[Alpha]: Hybrid Search용 파라미터

- 의미론적 검색과 키워드 기반 사이의 가중치(0 ~ 1 사이)
- 0이면 only 키워드 검색(BM25), 1이면 only 의미론 검색

[Fetch K]: MMR 검색용 파라미터

- K: 다양성을 고려할 후보(유사한) 청크의 개수

[Lambda]: MMR 검색용 파라미터

- Lambda(λ): 다양성과 관련성 사이의 가중치(0~1 사이)
- 0이면 다양성 중심, 1이면 관련성 중심 검색

쿼리 증강 : 사용자의 의도를 보다 정확하게 전달하기 위해 쿼리를 확장하는 과정

이슈 시나리오

Q1. 이번분기 삼성전자 실적 어때?

⇒ {query : "이번 분기 삼성전자 실적"}

A1. [생성한 질의로 문서 검색 후 답변]

Q2. 그럼 현대자동차는?

A2. 현대자동차에 대한 일반적 답변

쿼리 증강 시나리오

Q1. 이번분기 삼성전자 실적 어때?

⇒ {query : "이번 분기 삼성전자 실적"}

A1. [생성한 질의로 문서 검색 후 답변]

Q2. 그럼 현대자동차는?

채팅 히스토리를 고려하는 쿼리 재작성 프롬프트

Q2-1. 그럼 현대자동차는?

⇒ {query: "이번 분기 현대자동차 실적"}

A2. 현대자동차 이번 분기 실적 관련 답변

시스템 프롬프트 예시

역할

당신의 역할은 Query Writer입니다.

목표

사용자가 입력한 자연어 질문(한국어 또는 영어)을 분석하여, 해당 사용 의도와 목표를 가장 잘 충족하는 사내 문서 검색용 질의문을 하나 생성합니다.

JSON 외에 어떤 설명, 주석, 추가 텍스트도 절대 포함하지 않습니다.

[참고] 현재 시간은

{{ \$flow.state.current_date }} 입니다.

요구사항(규칙)

출력 형식은 반드시 아래 예시처럼 유효한 JSON → {"query": "검색 질의"}
query에는 단 하나의 규정 검색 질의문만 포함합니다.

멀티 쿼리 : 하나의 질의를 여러 검색 쿼리로 확장해 검색 성능을 높이는 방법

이슈 시나리오

Q1. 삼성전자 요즘 어때?

⇒ {query : "삼성전자 요즘 어때?"}

A1. [해당 질의로만 문서 검색 후 답변]



해당 질의로만 검색 시,
정보의 다양성이 떨어질 수 있는
문제점 존재

멀티 쿼리 시나리오

Q1. 삼성전자 요즘 어때?



질의와 유사한 다양한 쿼리를 생성한 뒤 각 쿼리에 대
해 모두 검색하여 다양한 정보 획득

Q1-1. {queries: "
[이번 분기 삼성전자 실적],
[삼성전자 최신 정보],
[삼성전자 AI 도메인 투자],
"]}



A2. 현대자동차 이번 분기 실적 관련 답변

시스템 프롬프트 예시

역할

당신의 역할은 Multi Query 입니다.

목표

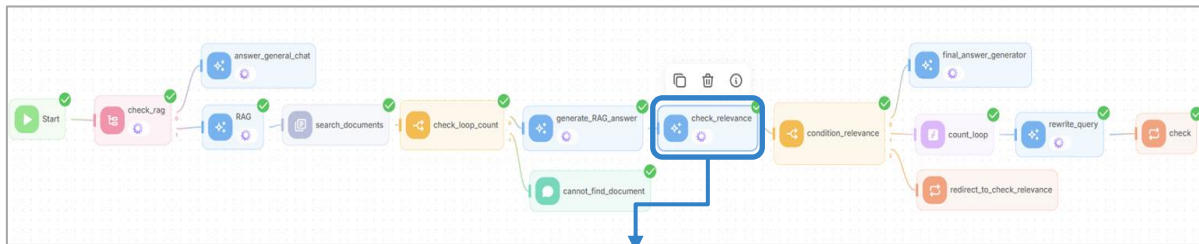
사용자가 입력한 자연어 질문(한국어 또는 영어)을 면밀히 분석하여, 해당 의도와 목표를 가장 잘 커버하는 다양한 주제의 질의를 생성해야 합니다.

출력은 오직 하나의 JSON 객체여야 하며, 키는 "queries"이고 값은 string 배열이어야 합니다. 다른 텍스트, 설명, 주석을 절대 포함하면 안됩니다.

요구사항(규칙)

출력 형식: 반드시 유효한 JSON. 최상위 키는 "queries" (예: {"queries": ["q1", "q2", ...]})로 작성하세요.

문서 적합성 워크플로우



시스템 프롬프트 예시

당신은 RelevanceChecker (관련성 평가자)입니다.
당신의 임무는 사용자 질문에 대해 생성된 답변이 **질문자가 요구한 실질적인 정보를 제공하는지**를 평가하는 것입니다.

🔍 **평가 기준 (모두 만족해야 "PASS")**

정보 포함 여부: 답변이 질문에 대한 **구체적이고 실질적인 정보**를 포함하고 있는가?

핵심 의도 해결: 답변이 질문의 **핵심적인 궁금증**이나 **요구사항**을 직접적으로 해결하고 있는가?

문맥 일치: 질문의 의도 및 맥락과 일치하는 정보를 제공하는가?

주제 이탈 여부: 주제에서 벗어나거나 다른 방향으로 흐르지 않았는가?

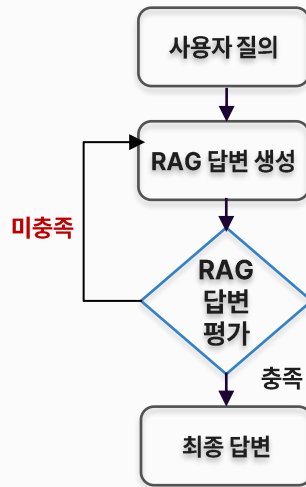
단, 질문의 핵심을 해결하지 못하면 NO

출력 형식

"PASS" 또는 "NO" 중 하나의 단어만 대문자로 출력

절대로 이유, 설명, 부연문장을 덧붙이지 말 것

상세 설명

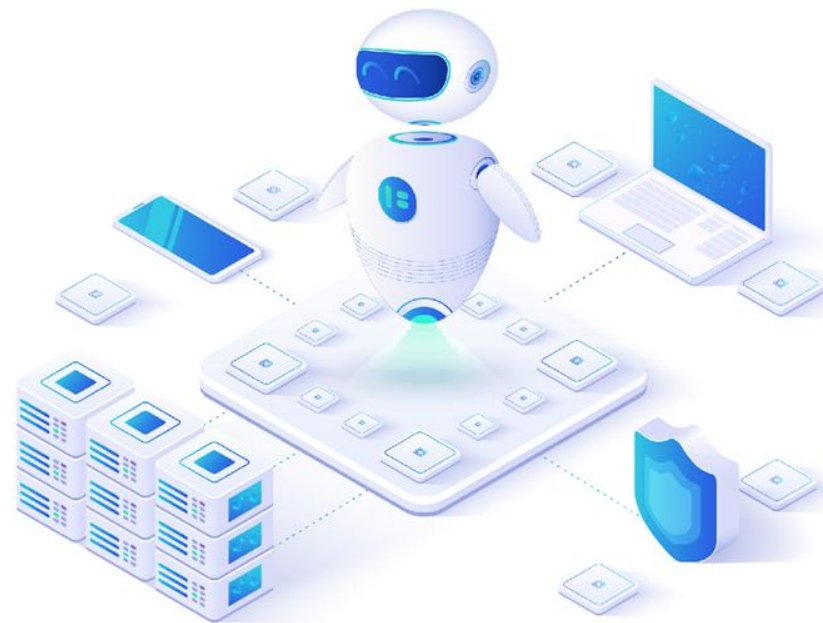


- 답변 생성 모델보다 한 단계 높은 성능의 모델을 활용하여 평가 신뢰도를 확보하는 것
권장

IV MCP

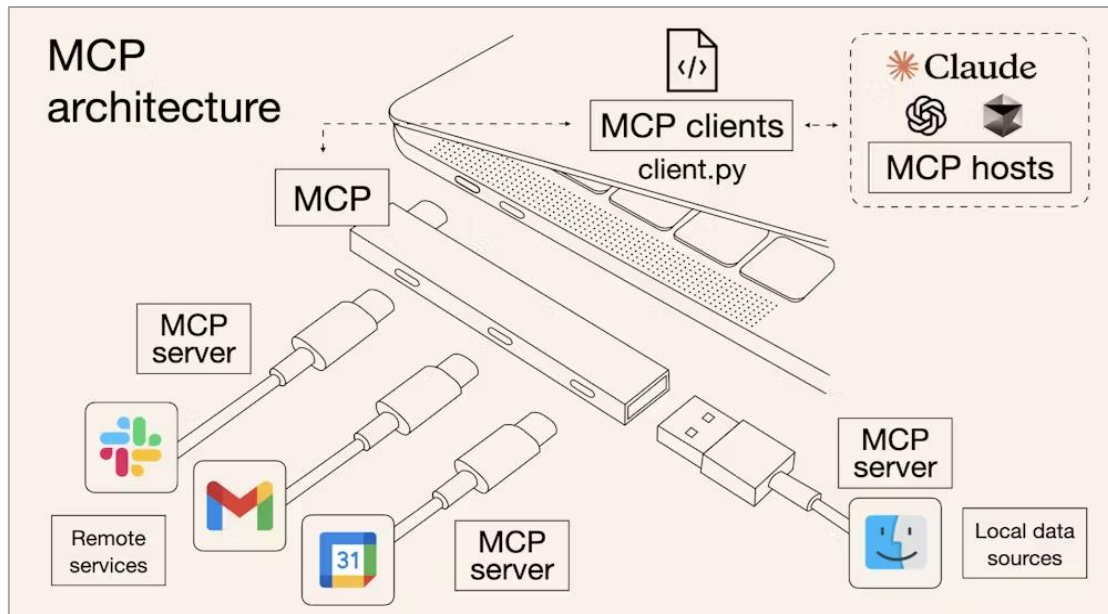
1. MCP 의 개념 43

2. GenOS MCP 44



Model Context Protocol

MCP 의 구성 요소



상세 설명

- **MCP의 등장 배경**
 - 애플리케이션별로 데이터 전달 방식 상이,
 - 새 도구가 등장할 때마다 매번 새로운 인터페이스 개발 필요
- **Model Context Protocol(MCP)**
 - LLM이 외부 컨텍스트를 전달 받는 방식을 통일함으로써 위 문제 해결
 - 전달 방식과 양식 같은 약속을 MCP로 통칭, 결국 MCP는 USB-C와 같은 규격을 말함
 - **MCP는 크게 Host, Client, Server로 구성**
- **MCP Host**
 - 사용자가 사용하는 AI 애플리케이션
 - MCP Host는 MCP Client 내장
 - ex) Claude, GPT, Cursor, Agent 노드 등
- **MCP Client**
 - MCP Host와 MCP Server를 연결
 - MCP Client의 동작을 사용자가 직접 관리 가능
- **MCP Server & Tool**
 - MCP Server: AI가 사용가능한 리소스 모음
 - MCP Tool: Server가 제공하는 구체적인 기능

GenOS에서 MCP 활용하기 : Flowise 노드

Flowise 노드

GenOS MCP 활용 절차

MCP 섹션

MCP 도구
생성MCP 서버
생성 및 배포

워크플로우 활용

GenOS MCP 관련 노드

Genos MCP

Inputs

Genos MCP Server *

사용가능한 도구 *

Output

Genos MCP Tool

Multiagent 3.0.0

Tool Node

Tool *

Genos MCP

Genos MCP Parameters

Genos MCP Server *

사용가능한 도구 *

Agentflow 3.0.0

상세 설명

• GenOS MCP

- GenOS에 등록된 MCP Server를 활용하는 노드
- Multiagent 3.0.0의 GenOS MCP Node 선택
- Agentflow 3.0.0 Tool 노드의 Tool로 선택

• GenOS MCP의 특징

- 다양한 툴을 파이썬 코드로 작성 가능
- 등록된 툴을 그룹화한 뒤 MCP 서버로 배포 가능
- 배포된 MCP 서버는 상술한 노드의 드롭다운에서 선택 가능

Tool *

Genos MCP

Genos MCP Parameters

Genos MCP Server *

사용가능한 도구 *

web-search - WEB_SEARCH

GenOS MCP 사용 예시

GenOS에서 MCP 활용하기 : MCP 도구 생성

MCP 도구 생성

MCP 도구 목록

ID	제목
191	[PubChem] substructure_search
7	지정된 길이의 랜덤 펩티드 시퀀스를 생성
4	텍스트를 분석하여 통계를 반환
1	현재 시간 조회

전체 4개 중 1 ~ 4 표시 / 페이지 표시 행수: 10

MCP 도구 생성

제목*
제목을 입력하세요

상세 설명
상세 설명을 입력하세요

관리 그룹*
관리 그룹을 선택하세요

파커지
파커지를 선택하세요

코드*

```
1 @mcp.tool()  
2 * async def tool_name(param1: str) -> str:  
3     """  
4     여기에 툴 설명을 작성하세요.  
5  
6     Args:  
7         param1 (str): 파라미터 설명  
8  
9     Returns:  
10        str: 반환 결과 설명  
11        # 여기에 문구를 작성하세요.  
12  
13        return "결과값"  
14  
15
```

닫기 생성

상세 설명

• MCP 도구 탭 접근

- 관리자 페이지 > 배포 > 도구 > MCP 도구
- GenOS에 등록된 MCP Server를 활용하는 노드
- Multiaгент 3.0.0의 GenOS MCP Node 선택
- Agentflow 3.0.0 Tool 노드의 Tool로 선택

• GenOS MCP의 특징

- 다양한 툴을 파이썬 코드로 작성 가능
- 툴에 대한 docstring을 통해 LLM 각각의 툴들이 어떤 역할을 하는지 파악 가능, docstring을 최대한 자세히 적는 것이 좋음
- 등록된 툴을 그룹화한 뒤 MCP 서버로 배포 가능
- 배포된 MCP 서버는 상술한 노드의 드롭다운에서 선택 가능

GenOS에서 MCP 활용하기 : MCP 서버 생성

MCP 서버 생성

도구 > MCP 서버

MCP 서버 목록

ID	제목	상세 설명
131	Chembi-MCP-Server2	
1	MCP 서버	

전체 2개 중 1 ~ 2 표시 / 페이지 표시 행수: 10

MCP 서버 생성 +

MCP 서버 생성

제목*
제목을 입력하세요

상세 설명
상세 설명을 입력하세요

관리 그룹*
GENON 시업개발팀 - 시연용

도커 이미지*
mnclmcp-server

인스턴스 타입*
smicro

MCP 도구*
MCP 도구를 선택하세요

저장된 이미지 컨테이너 레지스트리를 생성 ☐ 엑스트로 분석하여 통계를 반환 ☐

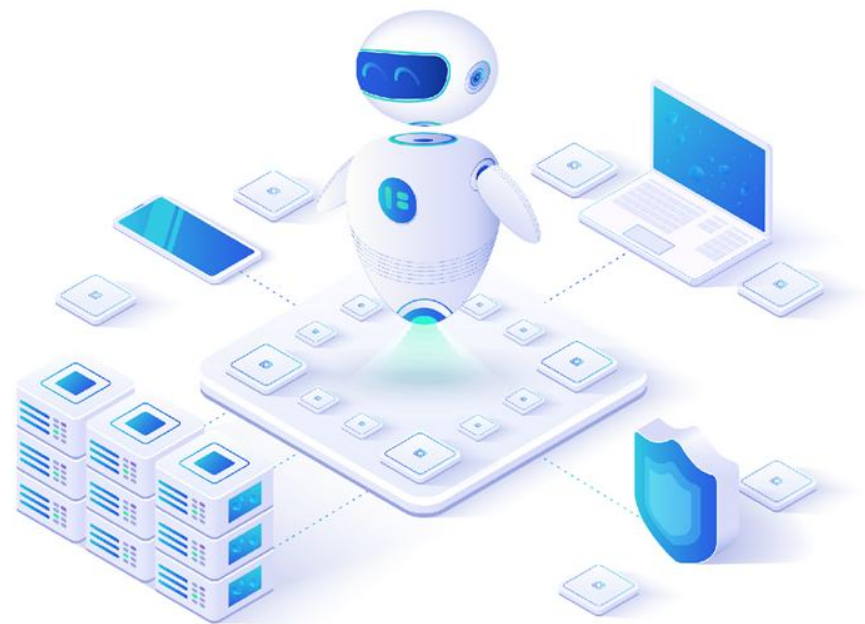
복제본*
1

닫기 생성

상세 설명

- **GenOS MCP 서버**
 - GenOS 관리자 페이지 > 배포 > MCP 서버 접속
- **GenOS MCP 서버 생성**
 - 제목 / 관리그룹
 - 도커 이미지, 인스턴스 타입 사용 도구 선택

V 참고자료



공식 문서 주소

리소스 유형	설명	URL
Flowise	• 공식 문서	• https://docs.flowiseai.com/
MCP	• 공식 문서	• https://modelcontextprotocol.io/docs/getting-started/intro
LangChain	• 공식 문서	• https://docs.langchain.com/
GenOS	• GenOS 공식 매뉴얼	• https://genos-docs.gitbook.io/default/v1.7.1
	• Flowise 노드 설명	• https://genos-docs.gitbook.io/default/v1.7.1/flowise-nodes/components/agentflow3-1-intro
	• RAG 워크플로우 만들기	• https://genos-docs.gitbook.io/default/v1.7.1/basic-tutorials/guides/workflow/agentflow3-2-rag
	• 워크플로우 제작 동영상 튜토리얼	• https://genos-docs.gitbook.io/default/v1.7.1/advanced-tutorials/undefined/undefined/undefined
	• GenOS API	• https://genos-docs.gitbook.io/default/v1.7.1/advanced-tutorials/guides/api/workflow
	• GenOS MCP	• https://genos-docs.gitbook.io/default/v1.7.1/basic-tutorials/guides/mcp/mcp-tool-server

감사합니다